Three Small Language Models, One Yardstick

A uniform evaluation of checkpoints A, B and C, with GPT-3 and open baselines

Model comparison report

November 2026

Abstract

We compare three decoder-only Transformer language models trained from scratch on a workstation with two RTX 2070 Super GPUs: A, a 54.5 M-parameter GPT-2-style model pre-trained on a small mixed-domain corpus; B, a 100.7 M-parameter Llama-style model taken through pre-training, mid-training and a full post-training pipeline (SFT, DPO, GRPO, self-distillation); and C, a 343.7 M-parameter model of the same family, put through the same pipeline at larger scale. Because the models use three different tokenizers, we score language-modelling quality in *bits per byte* of identical raw text, and we run every model through the same zero-shot benchmark harness. C is the best model on every quality metric except A's own lower-cased test set: mean 0.991 bpb against 1.052 for B and 1.778 for A, and HellaSwag 35.7% against 33.6% and 28.4%. B gets about 92% of C's bpb gain over A with less than a third of its parameters and about a fifth of its pre-training compute. Compared with the original GPT-3 family, C matches or beats GPT-3 Small on three of four zero-shot tasks with about 1/18 of its training compute. Against open models of similar size (GPT-2, OPT, Pythia, SmolLM2) measured with the same harness, B and C are competitive per FLOP but trail the models trained on far more tokens. A is the cheapest model by one to two orders of magnitude, but it is barely above chance on reasoning benchmarks, cannot model case or code well, and is limited to a 512-token context.

1 Models under test

	A	B	C
Checkpoint	`a.pt`, step 60 000 (epoch 10)	`b.pt`, final post-trained model	`c.pt`, final post-trained model
Lineage	`QPT-LLM-prototype-no-lib`	`llm_pipeline/work` ("100M" run)	`llm_pipeline/work_350m`
Layers / d_{model} / heads	12 / 512 / 8 (MHA)	12 / 768 / 12 (4 KV heads, GQA)	28 / 1024 / 16 (4 KV heads, GQA)
Block	pre-norm LayerNorm, GELU MLP ($4d$), biases	pre-norm RMSNorm, SwiGLU (d_{ff}=2048)	pre-norm RMSNorm, SwiGLU (d_{ff}=2752)
Positions	learned absolute, 512 max	RoPE + YaRN-style scaling, 4096	RoPE + YaRN-style scaling, 4096
Unique parameters	54.5 M (tied emb.)	100.7 M (tied emb.)	343.7 M (tied emb.)
Tokenizer	BPE 32k, *lower-cases and strips accents*	byte-level BPE 32 768	byte-level BPE 32 768 (separate training)
Pre-training data	9 HF corpora (news, reviews, Wikipedia, Q&A); 37 M unique tokens, 10 epochs	web 74%, code 12%, math 8%, wiki 6%	same mix, about 2.3× more text
Tokens seen (pre-train)	0.38 B	3.42 B	5.46 B
Tokens per parameter	7	34	16
Post-training	none (base LM)	mid-train → SFT → RM → DPO → GRPO → distill	same

Table 1: The three checkpoints. Parameter counts are unique tensors (the tied input/output embedding is counted once).

Provenance note. The `tokens_c/` directory shipped next to `c.pt` holds the tokenizer of the pipeline's *smoke-test* run, with a vocabulary of 4 096 (its MD5 hash matches `work_smoke`). It cannot encode text

for C, whose embedding table has 32 768 rows. All results for C therefore use the tokenizer of the run that produced it, `work_350m/3_tokenization/tokenizer.json`. `a.pt` matches A's training checkpoint `ckpt_step_0060000.pt` byte for byte.

2 Methodology

Tokenizer-independent perplexity. Per-token loss cannot be compared across different vocabularies, so we report bits per byte:

$$\text{bpb} = \frac{\sum_t -\ln p(x_t \mid x_{<t})}{\ln 2 \cdot N_{\text{bytes}}},$$

where N_{bytes} is the UTF-8 length of the *original* text. Each model re-tokenizes the same documents with its own tokenizer. Every document is scored on its own, starting from BOS, in non-overlapping windows of 512 tokens (A's maximum context). B and C are also scored with 2 048-token windows. The text comes from five held-out sets of about 0.55–0.67 MB each: the web, code, math and Wikipedia validation shards of the B pipeline, and A's own test split ("A-test", which was lower-cased by A's preprocessing).

Check: A scores 3.954 nats/token on A-test with this harness, against 3.957 in its training log, so the reimplementation of A is faithful. We found that A was trained on single documents padded to 512 tokens (only 39% of its training tokens are real text). Scoring it on packed multi-document windows would therefore be unfair to it, and that is why every document is scored on its own.

Zero-shot benchmarks. HellaSwag, ARC-Easy and PIQA are scored by the log-likelihood of each answer, using the pipeline's own harness logic (`eval/benchmarks.py`). We report length-normalised accuracy (acc_norm) for HellaSwag and PIQA and raw accuracy for ARC-Easy. LAMBADA requires an exact greedy prediction of the final word. Each benchmark uses the first 2000 items, or 1 838 for PIQA, and the error bars are 95% Wilson intervals. All three models are run on the same items with the same prompts. A's inputs are left-truncated to 512 tokens.

Efficiency. We measure fp16 forward throughput at sequence length 512 (batch sizes 1 and 8), greedy decode latency without a KV cache (identical for all models), peak allocated VRAM, and checkpoint size, all on one RTX 2070 Super. Training compute, energy and wall-clock time come from the training logs. For A, compute is estimated as $6ND$ over non-pad tokens.

3 Results

3.1 Language-modelling quality

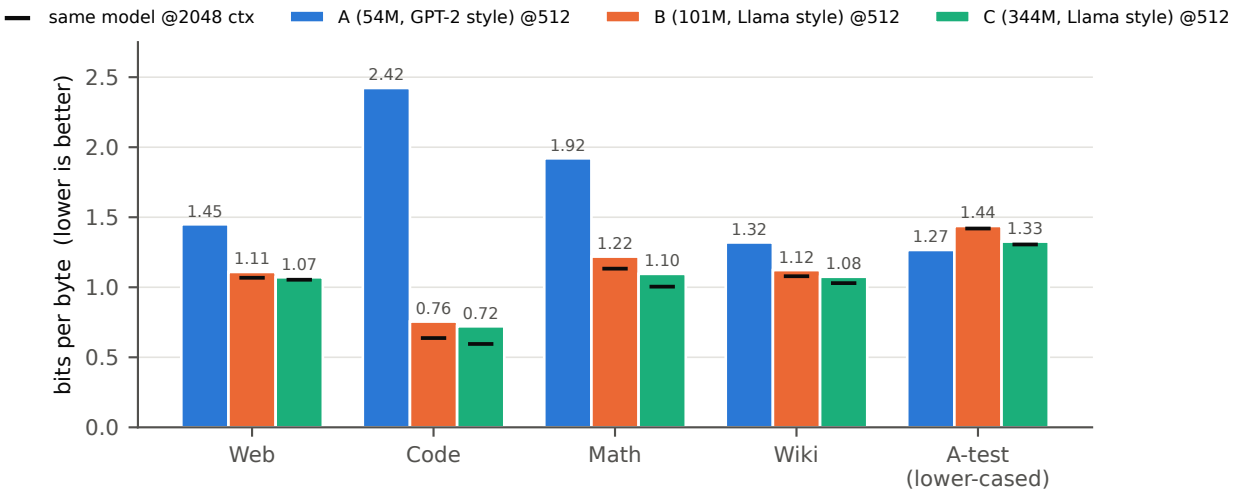


Figure 1: Bits per byte on identical raw text, using 512-token windows (bars). Black ticks show B and C with 2 048-token windows. Lower is better.

Figure 1 and Table 2 show four patterns:

Model / context	Web	Code	Math	Wiki	A-test
A @ 512	1.450	2.423	1.921	1.320	1.266
B @ 512	1.109	0.755	1.219	1.123	1.438
B @ 2048	1.068	0.637	1.133	1.079	1.419
C @ 512	1.071	0.721	1.096	1.075	1.326
C @ 2048	1.054	0.595	1.004	1.030	1.305
A bytes/token	4.62	3.05	3.47	4.38	4.50
B bytes/token	4.41	3.81	3.56	3.89	4.01
C bytes/token	4.40	3.79	3.56	3.88	4.01

Table 2: Bits per byte by domain and context length. The bottom block shows tokenizer compression (bytes per token, higher means fewer tokens for the same text).

- **Size and data order the models consistently.** C < B < A on all four pipeline domains. The mean over the four pipeline domains is 0.991, 1.052 and 1.778 bpb.
- **A's biggest gap is code** (2.423 vs. 0.721 bpb). Its corpus has essentially no source code, and its normaliser lower-cases identifiers and strips accents, so its tokenizer fragments code into short pieces (about 3 bytes/token, against 3.8 for B and C).
- **A-test is A's home ground, and A wins there.** The text is already lower-cased and comes from A's own distribution. A scores 1.266, ahead of C (1.326) and B (1.438). On in-distribution text, a small model trained on that data beats much larger general-purpose models.
- **Long context helps B and C.** Scoring with 2 048-token windows lowers mean bpb to 0.979 (B) and 0.921 (C). The largest gains are on code and math, where long-range structure matters. A cannot use context beyond 512 tokens at all.

3.2 Zero-shot benchmarks

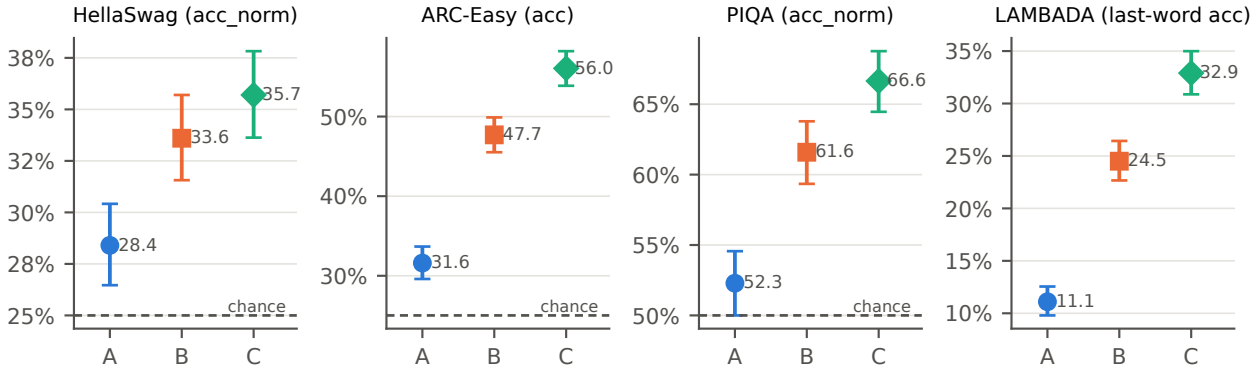


Figure 2: Zero-shot accuracy with 95% Wilson intervals. Dashed lines mark chance.

On the benchmarks (Figure 2), A sits a few points above chance on HellaSwag (28.4%) and ARC-Easy (31.6%), and it is barely above chance on PIQA (52.3%, with the interval's lower edge at about 50%). C is clearly ahead on ARC-Easy (56.0%) and PIQA (66.6%), and leads strongly on LAMBADA (32.9% against 24.5% and 11.1%). LAMBADA tests long-range narrative tracking, which is exactly where depth and data help. On HellaSwag, B (33.6%) and C (35.7%) are close. At this scale HellaSwag rises slowly, as Figure 3 (right) shows. LAMBADA perplexity separates the models most sharply: 710 for A, 22 for B and 11 for C.

3.3 Training dynamics, data and compute

Figure 4 gives the economics:

- **Compute.** Pre-training cost about 1.25×10^{17} FLOPs for A, 2.26×10^{18} for B and 1.22×10^{19} for C.
- **Scaling.** Mean bpb falls roughly as a power law in compute. A, though, is off the B–C trend: its architecture is older, its corpus is narrow and lower-cased, and it repeated the same 37 M tokens ten times.
- **Energy.** Measured end-to-end training energy, from pre-training through post-training, is 0.1 kWh for A, 10.5 kWh for B and 99.5 kWh for C. C's mid-training and full-length SFT cost almost as much energy as its pre-training.

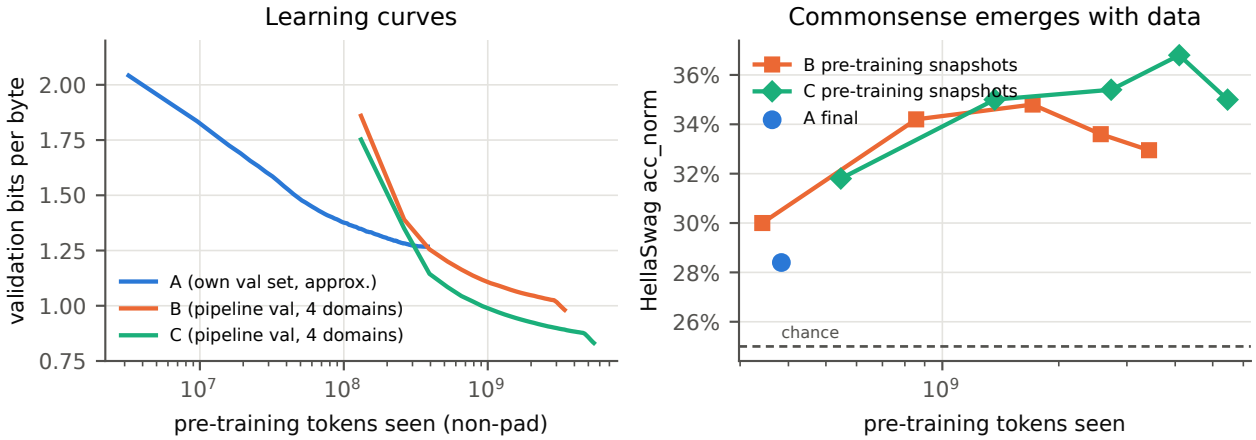


Figure 3: Left: validation bpb during pre-training, read from the logs. B and C were evaluated on the pipeline's 4-domain validation set. A's curve is its own validation loss converted to bpb using its measured 4.48 bytes/token, so it is approximate and on a different text distribution. Right: HellaSwag accuracy of B and C at pre-training snapshots (10%, 25%, 50%, 75%, 100%), with A's final checkpoint for reference.

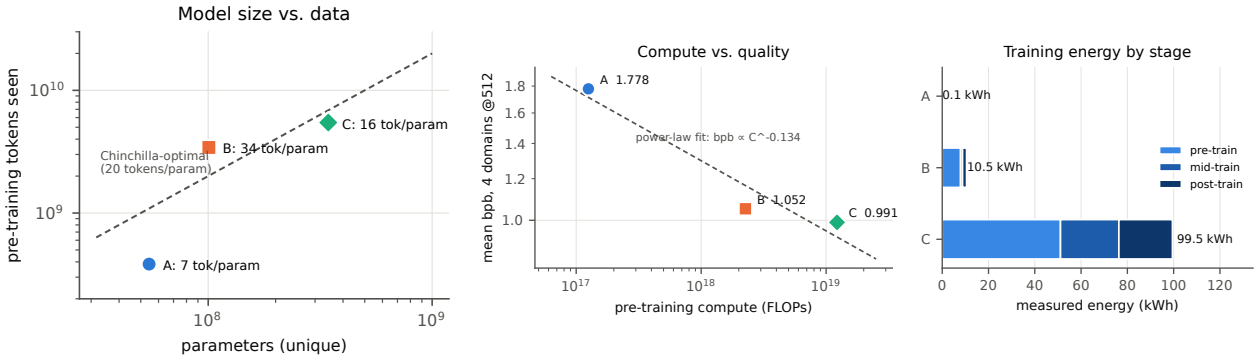


Figure 4: Left: parameters vs. pre-training tokens. A saw only 7 tokens per parameter, cycling 10 times over 37 M unique tokens. B was trained past the Chinchilla ratio, and C sits just below it. Centre: pre-training compute vs. uniform mean bpb, with a power-law fit through the three points. Right: measured training energy by stage.

- **Caveat on A's energy.** A's figure is the prototype's own GPU-side meter over 7.1 h. It is not directly comparable with the pipeline's whole-system estimate, and likely understates A's true energy.

3.4 Inference efficiency

A processes about 142k tokens/s at batch 8, against 66k for B and 27k for C. Its greedy decode takes 6.6 ms/token, against 15.0 and 31.2 ms, and it fits in 0.58 GB, against 0.86 and 2.21 GB (Figure 5). B sits at the knee of the curve. It is about 2.4× faster than C and costs only about $1.052 - 0.991$ bpb.

3.5 Inside the networks

All three models depend most on their first block: skipping it costs 1.4–2.2 bpb (Figure 6).

- **A and C** also depend heavily on block 1 and on the last block.
- **B** spreads its computation more evenly, at about 0.05–0.14 bpb per block.
- **C's middle layers are redundant.** Many of them cost less than 0.03 bpb when removed, and a few slightly *improve* bpb, making C a natural candidate for depth pruning or early exit.
- **Weight scales differ by family.** The Llama-style models keep their weights roughly flat across depth, apart from a larger first block, consistent with weight decay 0.1 and scaled residual initialisation. A's weights grow steadily with depth.

3.6 What post-training and mid-training bought (B and C)

The pipeline logs (Figures 7–8) show that post-training transformed what C can do:

- **Math.** Synthetic arithmetic accuracy went from about 1% to 99.6%, and GSM8K from 2% to 17.6%.

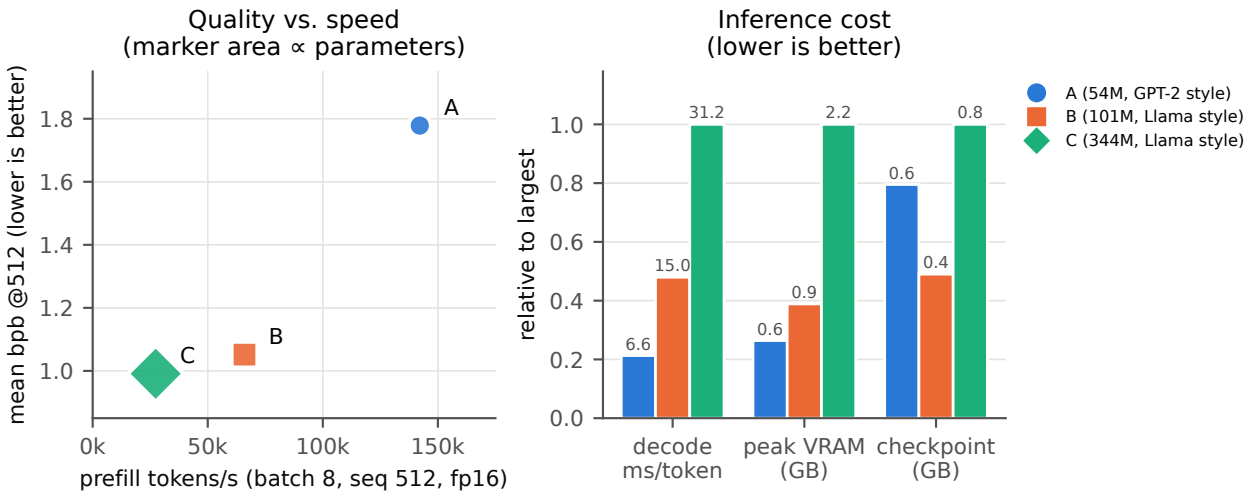


Figure 5: Left: quality vs. prefill throughput. Right: inference cost, normalised to the most expensive model, with absolute values printed above the bars.

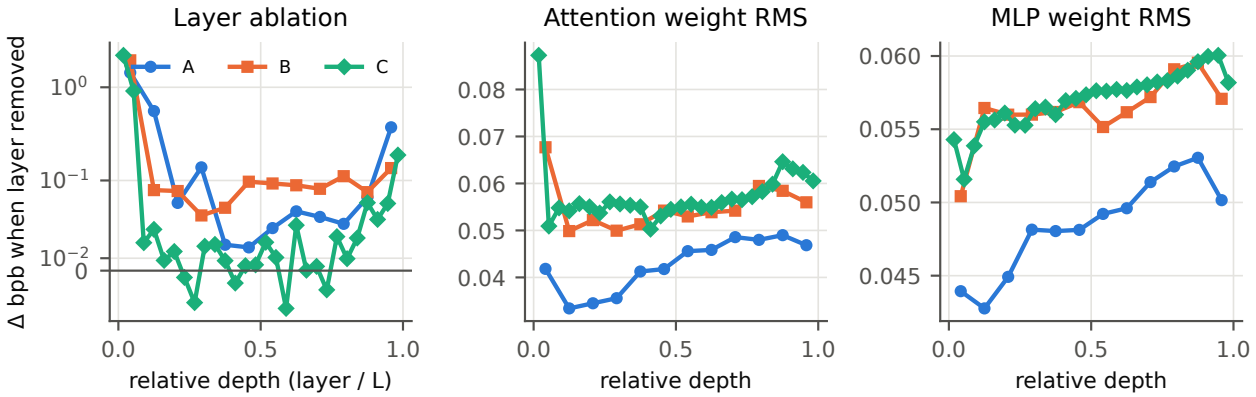


Figure 6: Left: increase in bpb when one whole block is skipped (web + wiki text, symmetric log scale). Middle and right: RMS of each block's attention and MLP weights, plotted against relative depth.

- **Code.** HumanEval pass@1 went from 0.6% to 9.1%.
- **Long context.** Needle retrieval went from failing beyond 1k tokens to about 90–100% up to 4k.
- **Conversation.** Multi-turn fact recall reached about 70% after distillation.

The cost was a small rise in raw-text bpb: the pipeline's own log reports 0.82 at the base checkpoint and 0.92 after SFT, a known effect of instruction tuning. B's pipeline did not evaluate its post-trained stages, so the hollow markers are its only post-training measurement. A has no instruction or chat ability.

3.7 Qualitative samples

Table 5 lists greedy continuations.

- **A** produces fluent, lower-case prose but quickly falls into repetition loops.
- **B and C** follow the chat template.
- **B and C** both solve the arithmetic chat prompt ($12 \times 7 = 84$), but only **C** gives a coherent explanation of why the sky is blue. B loops on the wording of the question.
- **Base-mode facts** are shaky for all three: only B and C complete "The capital of France is" with Paris, and only C gets close on the boiling point of water.

4 Comparison with GPT-3 and contemporary small models

To place A, B and C in context, we compare them with two groups:

- **GPT-3** (Brown et al., 2020), the original 2020 model family. Its weights were never released, so we take the zero-shot numbers *published* in the paper's Table H.1. All GPT-3 sizes were trained on 300 B tokens with a 2 048-token context. We checked these numbers against the paper; they were not re-measured.

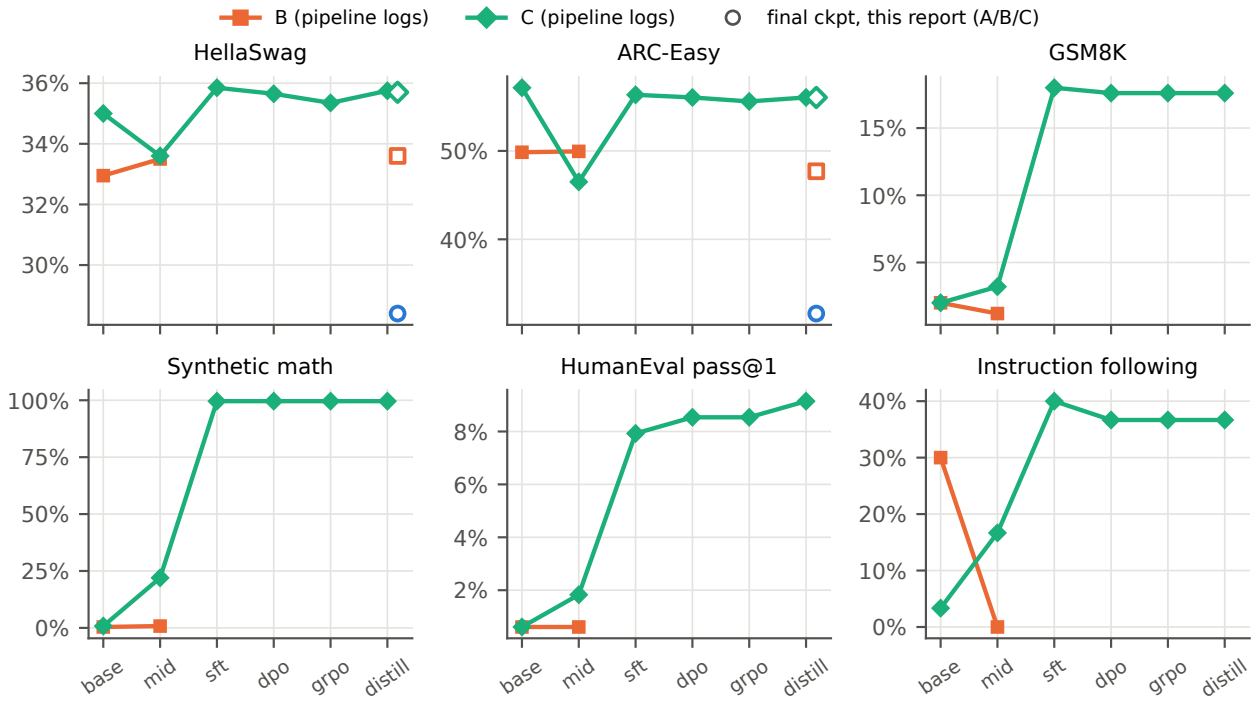


Figure 7: Capability across pipeline stages, from the pipeline's own evaluation logs. Only base and mid-training were evaluated for B. Hollow markers show the final checkpoints measured in this report, on the same items.

- **Open models of similar size**, run through *exactly the same harness* as A, B and C: the same raw text for bpb, the same benchmark items, prompts and scoring. These are GPT-2 124M and 355M (2019), OPT-125M (2022, 180 B tokens), Pythia-160M and 410M (2023, 300 B tokens on the Pile), and SmolLM2-135M and 360M (2024, 2 T and 4 T tokens).

Model	Year	Params	Train tok.	Ctx	HellaSw.	ARC-E	PIQA	LAMB. acc	LAMB. ppl	bpb
GPT-3 Small†	2020	125M	300B	2048	33.7	43.6	64.6	42.7	18.6	–
GPT-3 Medium†	2020	356M	300B	2048	43.6	46.5	70.2	54.3	9.1	–
GPT-3 Large†	2020	760M	300B	2048	51.0	53.0	72.9	60.4	6.5	–
GPT-3 XL†	2020	1,320M	300B	2048	54.7	53.8	75.1	63.6	5.4	–
GPT-3 175B†	2020	174,600M	300B	2048	78.9	68.8	81.0	76.2	3.0	–
GPT-2 124M	2019	124M	n/r	1024	34.8	44.5	62.2	34.5	15.2	1.243
GPT-2 355M	2019	355M	n/r	1024	38.6	49.4	67.3	44.1	8.9	1.107
OPT 125M	2022	125M	180B	2048	34.5	43.2	61.8	39.4	10.9	1.194
Pythia 160M	2023	162M	300B	2048	33.7	44.3	61.8	34.8	12.8	1.024
Pythia 410M	2023	405M	300B	2048	38.2	52.8	68.2	51.2	5.5	0.885
SmolLM2 135M	2024	135M	2T	2048	38.9	64.3	68.6	43.8	7.8	0.895
SmolLM2 360M	2024	362M	4T	2048	43.5	70.3	72.7	54.4	5.0	0.796
A (this work)	2026	54M	0.38B	512	28.4	31.6	52.3	11.1	709.8	1.778
B (this work)	2026	101M	3B	4096	33.6	47.7	61.6	24.5	22.5	1.052
C (this work)	2026	344M	5B	4096	35.7	56.0	66.6	32.9	10.6	0.991

Table 3: Zero-shot comparison. †GPT-3 rows are published values (HellaSwag and PIQA on dev, ARC-Easy and LAMBADA on test, full sets). All other rows were measured here on the same items: the first 2 000 of HellaSwag (dev), ARC-Easy (test) and LAMBADA-OpenAI, and all 1 838 PIQA dev items. HellaSwag and PIQA are length-normalised accuracy. "bpb" is the mean over the four pipeline domains at 512 tokens. Training-token counts are as reported by each model's authors (n/r = not reported).

C vs. GPT-3. C is about the size of GPT-3 Medium (350M) but was trained on 55× fewer tokens and with 18× less compute than even GPT-3 *Small*.

- **Against GPT-3 Small (125M),** C matches or beats it on three of four tasks: HellaSwag 35.7 vs. 33.7, ARC-Easy 56.0 vs. 43.6, and PIQA 66.6 vs. 64.6. It falls well short on LAMBADA (32.9 vs. 42.7).
- **Against GPT-3 Medium (350M),** C trails on HellaSwag (43.6), PIQA (70.2) and LAMBADA (54.3), but is ahead on ARC-Easy (46.5).

Modern data mixes rich in educational, code and math text explain most of the ARC-Easy gain. The remaining gaps track the 55× smaller token budget.

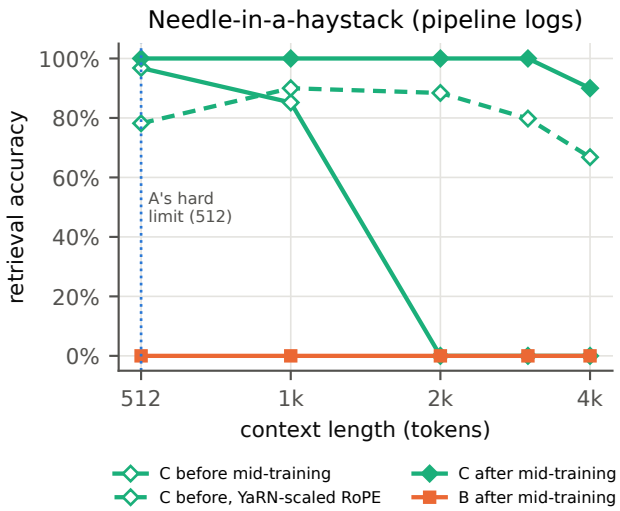


Figure 8: Needle-in-a-haystack retrieval, from the pipeline logs. Mid-training at 1k→4k tokens with rescaled RoPE gave C near-perfect retrieval up to 4 096 tokens. B never retrieved the needle.

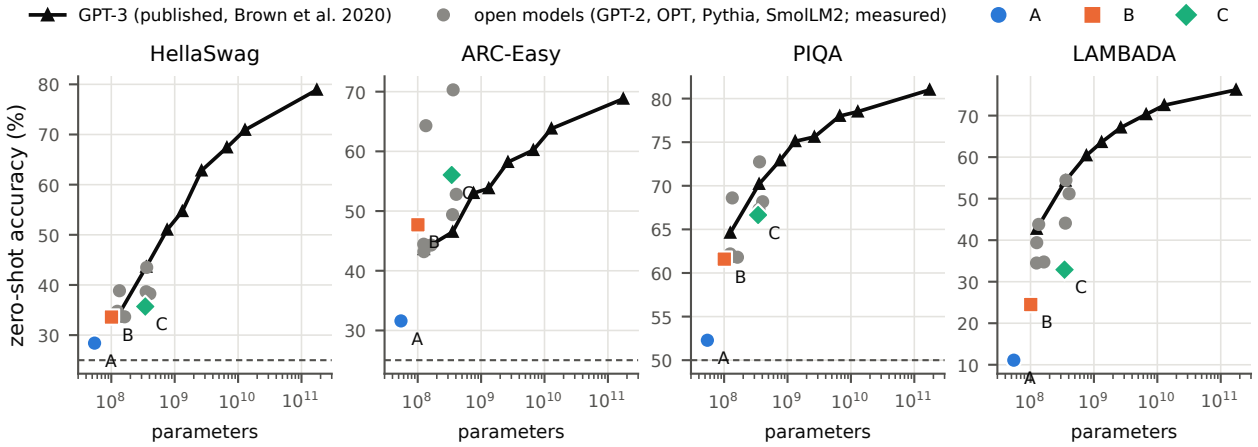


Figure 9: Zero-shot accuracy vs. parameter count. The black line is the GPT-3 family (published). Grey points are open models measured with our harness. Coloured markers are A, B and C.

A, B, C vs. open models of the same size.

- **B vs. GPT-2 124M** (which it slightly undercuts in size): B has better bpb (1.052 vs. 1.243) and ARC-Easy (47.7 vs. 44.5), similar PIQA (61.6 vs. 62.2), but lower HellaSwag (33.6 vs. 34.8) and LAMBADA (24.5 vs. 34.5).
- **C vs. GPT-2 355M,** the closest 2019 model in size: C has better bpb (0.991 vs. 1.107) and ARC-Easy (56.0 vs. 49.4), and is behind on HellaSwag (35.7 vs. 38.6) and LAMBADA (32.9 vs. 44.1).
- **Pythia-410M and SmolLM2-360M,** trained on 55–730× more tokens than C, are ahead of C on every metric except Pythia's ARC-Easy. SmolLM2-360M is the strongest model measured: bpb 0.796, HellaSwag 43.5, ARC-Easy 70.3, LAMBADA 54.4. Even SmolLM2-135M, at under half of C's size, beats C on every metric (bpb 0.895, HellaSwag 38.9, ARC-Easy 64.3).
- **A** is below every open model on every metric. The smallest of them, OPT-125M, still has more than twice A's parameters and was trained on almost 500× more tokens.

What this means. Figure 10 shows that, at this scale, *training tokens* matter more than architecture:

- Among models of comparable size, quality follows the data budget: A (0.4 B) < B (3.4 B) < C (5.5 B) < Pythia/OPT (180–300 B) < SmolLM2 (2–4 T).
- LAMBADA is the most data-hungry benchmark. C's LAMBADA score was still rising steeply during pre-training (14% at 10% of training to 31% at the end, from the pipeline's snapshot evaluations), and post-training did not change it. B and C are therefore under-trained relative to their capacity, rather than limited by their design.
- For their *compute*, B and C are competitive. C reaches GPT-3-Small-level commonsense scores with about

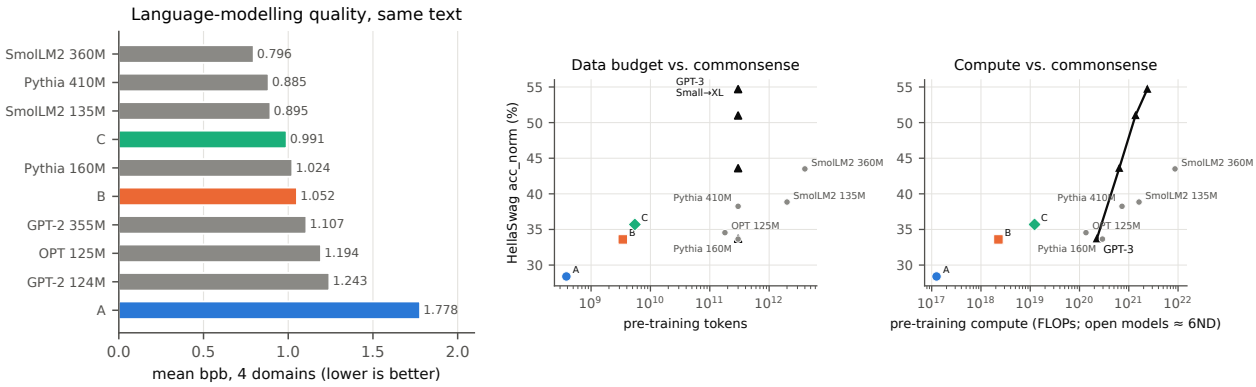


Figure 10: Left: mean bits per byte on identical text for all ten measured models. Centre and right: HellaSwag against pre-training tokens and against pre-training compute. Compute for open models is estimated as $6ND$; GPT-3 compute is from the paper's Table D.1.

5% of GPT-3 Small's training FLOPs, on a two-GPU workstation.

Caveats specific to this comparison.

- **Different scoring protocol for GPT-3.** The GPT-3 numbers use the paper's own protocol: full test sets, and for ARC an answer normalisation by unconditional probability. They are therefore not strictly comparable with the measured rows.
- **Different data for each model.** The bpb texts come from the B/C pipeline's sources, which favours models trained on similar web, code and math mixes (B, C, SmolLM2). GPT-2, OPT and Pythia never saw those exact distributions.
- **Precision.** SmolLM2-360M was trained in bf16 and overflows in fp16, so it was evaluated in fp32. Its throughput is therefore not comparable with the other models.

5 Overall comparison

Scorecard (cell shade = relative standing, darker = better)

	A (54M, GPT-2 style)	B (101M, Llama style)	C (344M, Llama style)
Mean bpb @512 ↓	1.778	1.052	0.991
Web bpb ↓	1.450	1.109	1.071
Code bpb ↓	2.423	0.755	0.721
A-test bpb ↓	1.266	1.438	1.326
HellaSwag ↑	28.4%	33.6%	35.7%
ARC-Easy ↑	31.6%	47.7%	56.0%
PIQA ↑	52.3%	61.6%	66.6%
LAMBADA ↑	11.1%	24.5%	32.9%
Throughput ↑	142k tok/s	66k tok/s	27k tok/s
Peak VRAM ↓	0.58 GB	0.86 GB	2.21 GB
Train energy ↓	0.1 kWh	10.5 kWh	99.5 kWh
Context window ↑	512	4096	4096

Figure 11: Scorecard. Each row is min–max normalised, darker = better; raw values are printed in the cells.

Ranking.

1. C — best quality on every metric except A-test, the only model with reliable 4k-token context and usable math, code and chat skills. It costs the most to train, at roughly 9.5× the energy of B, and is the slowest to serve.

Metric	A	B	C
Mean bpb @512 ↓	1.778	1.052	**0.991**
Web bpb ↓	1.450	1.109	**1.071**
Code bpb ↓	2.423	0.755	**0.721**
A-test bpb ↓	**1.266**	1.438	1.326
HellaSwag ↑	28.4%	33.6%	**35.7%**
ARC-Easy ↑	31.6%	47.7%	**56.0%**
PIQA ↑	52.3%	61.6%	**66.6%**
LAMBADA ↑	11.1%	24.5%	**32.9%**
Throughput ↑	**142k tok/s**	66k tok/s	27k tok/s
Peak VRAM ↓	**0.58 GB**	0.86 GB	2.21 GB
Train energy ↓	**0.1 kWh**	10.5 kWh	99.5 kWh
Context window ↑	512	**4096**	4096

Table 4: Headline numbers (best value in bold).

2. B — the efficiency sweet spot. About 92% of C's bpb improvement over A at 29% of its parameters, 2.4× its throughput and a fraction of its training energy. Its weak points are long-context retrieval and LAMBADA.
3. A — a useful educational baseline and the fastest model, but data-starved (7 tokens/param, repeated for 10 epochs), case-insensitive, limited to 512 tokens and without instruction tuning. Its quality per FLOP is the lowest of the three.

Threats to validity.

- **A's lower-casing.** A predicts lower-cased, accent-stripped text while bpb is counted against the original bytes, which flatters A slightly.
- **Possible contamination.** B's and C's corpora come from the same sources, so B's validation shards may overlap C's training data. This would favour C on the four pipeline domains. On A-test, which neither B nor C trained on, C still beats B by about 0.11 bpb.
- **Small benchmark samples.** With $n \leq 2000$ items, differences under about 2 points are within noise (see the intervals in Figure 2).
- **Not comparable across runs.** Energy figures, and the stage results for B and C, come from each run's own logs and were not re-measured.

A Greedy samples

Table 5: Greedy samples from each model (↩ marks a newline).

Model	Mode	Prompt	Greedy continuation (64–96 tokens, truncated)
A	raw	`The capital of France is`	a french city in the province of saint-et-loire, in the province of saint-et-loire. it is located in the north of the city of saint-et-loire. history the capital of france was founded in the 16th century by the french gover
A	raw	`Once upon a time, in a small village by the sea,`	the film is a bit slow, but the film is a bit slow, and the acting is good. the film is a little slow, but the film is a little slow, and the acting is good. the film is a little slow, but it is a little slow, and the film is a li
A	raw	`def fibonacci(n):`	the supreme court on tuesday ordered the supreme court to allow the court to allow the court to allow the court to allow the court to allow the court to allow the court to allow the court to allow the court to allow the court to a
A	raw	`Water boils at a temperature of`	about 2.5 degrees celsius. the temperature of water is about 2.5 degrees celsius. the temperature of water is about 2.5 degrees celsius. the temperature of water is about 2.5 degrees celsius. the temperature of water is about

Table 5: Greedy samples from each model (↩ marks a newline).

Model	Mode	Prompt	Greedy continuation (64–96 tokens, truncated)
B	raw	`The capital of France is`	Paris. Paris is a city known for its rich history, culture, and art. The city is known for its rich history, culture, and art. Paris is a major center for art, fashion, and fashion, and it is also a major center for art, fashion,
B	raw	`Once upon a time, in a small village by the sea,`	a woman named Luna, with a beautiful smile, stood before a man named Luna, who had been walking among the trees. Luna, with a smile, said, "Luna, I am Luna, and I am the one who brings you to this place." Luna, with a smile,
B	raw	`def fibonacci(n):`	if n <= 1: return fibonacci(n-1) else: return fibonacci(n-2) # Driver Program n = 10000000000000000000000000
B	raw	`Water boils at a temperature of`	20°C, and then it reaches a temperature of 25°C. If the temperature of the water is 20°C, then the water boils at a temperature of 25°C. The total volume of water in the water is 20 + 25 = 45 liters
B	chat	`Explain why the sky is blue in two sentences.`	The sky is blue in two sentences. The first sentence is about a clear, clear sky, which is a clear, clear sky. The second sentence is about a clear, clear sky, which is a clear, clear sky. The third sentence is about a clear, clea
B	chat	`What is 12 * 7?`	10 * 7 = 70; 2 * 7 = 14; 70 + 14 = 84The answer is 84.
C	raw	`The capital of France is`	Paris. The current population of Paris is 271,088 in 2021. The average age of the population is 38.4 years. The standard deviation is 18.5 years. a) What is the probability that a randomly selected person from the population
C	raw	`Once upon a time, in a small village by the sea,`	there lived a kind and wise old man named Mr. Gatsby. He was known throughout the land for his love of literature and his wisdom on the subject of friendship. One day, a young girl named Lily walked into the village, curious about
C	raw	`def fibonacci(n):`	# calculate the nth Fibonacci number fib_n = 0 for i in range(2, n): fib_n = fib_n + fib_n return fib_n # Driver Program n = 10 print("Fibonacci Number", n, "is
C	raw	`Water boils at a temperature of`	100 degrees Fahrenheit and 100 degrees Celsius, respectively. If you pour water into a kettle that is at the same temperature as the kettle, how many times will the water boil before it reaches a temperature of 100 degrees Fahrenh
C	chat	`Explain why the sky is blue in two sentences.`	The sky appears blue because of the way sunlight interacts with the Earth's atmosphere. When sunlight reaches the Earth's surface, it collides with molecules in the atmosphere, causing the molecules to scatter the light in all dir
C	chat	`What is 12 * 7?`	10 * 7 = 70; 2 * 7 = 14; 70 + 14 = 84The answer is 84.

B Reproducing

```
cd report
python3 eval_compare.py --model A --gpu 1   # also B, C  -> results/*.json
python3 gather_logs.py                      # training/energy/stage logs -> results/logs.json
python3 make_figs.py                        # figs/*.pdf, results/*.tex
latexmk -pdf report.tex
```